
INSTITUT SUPÉRIEUR POLYTECHNIQUE DE MADAGASCAR

Fahaizana · Fampandrosoana · Fihavanana



HACKATHON AI ENGINEERING & GÉNÉRATION VECTORIELLE

IconForge AI

Générer une famille d'icônes SVG cohérente
à partir de concepts en langage naturel

Examen de fin d'études

Public : Master 2 — AI Engineering

Durée : 8 heures — de 08h00 à 16h00

Modalité : distanciel complet, équipes de 2 à 7 étudiants

Deadline : 16h00 précises — dernier commit

Livrable évalué : un fichier `<id>.svg` par concept soumis

Épreuve finale : exécution du dépôt sur des concepts et une charte inconnus

RABOANARY Heriniaina Andry

Charte graphique pédagogique créée pour l'épreuve, sans affiliation avec aucune marque.

1. Contexte et organisation

À propos du commanditaire

Une organisation souhaite automatiser la création d'icônes pour ses interfaces, ses documents et ses supports pédagogiques. Une simple collection d'images disparates ne suffit pas : chaque nouvelle icône doit représenter correctement son concept tout en conservant le langage visuel de la marque. Aujourd'hui, chaque nouvel écran demande un aller-retour avec un graphiste, et le résultat dérive d'un lot à l'autre.

Mission critique

Votre mission est de construire **IconForge AI**, un système capable de générer une famille d'icônes SVG cohérente à partir de concepts exprimés en langage naturel. Le système doit produire une icône *juste* — qui représente bien son concept — sans jamais sortir du langage visuel commun à la collection.

Le système devra produire un fichier SVG par concept demandé, mesurer lui-même la qualité de ses productions, et arbitrer explicitement entre fidélité au concept et fidélité au style.

Organisation de la journée

Durée	8 heures — 08h00 à 16h00
Modalité	Distanciel complet, équipes de 2 à 7 étudiants
Deadline	16h00 précises — le dernier commit antérieur à la deadline fait foi

2. Ressources fournies

Fichiers fournis

Fichier	Contenu
brand-guidelines.md	Charte graphique — <i>remplacée par le jury à l'évaluation</i>
references/*.svg	Cinq icônes de référence — <i>remplacées de même</i>
benchmark/public-concepts.json	Concepts publics et leurs contextes courts
benchmark/request.schema.json	Schéma d'entrée de l'adaptateur
submission-template/README.md	Canevas imposé du rapport
evaluation/rubric-public.md	Détail public des critères d'évaluation

Ce que fixe la charte

Les valeurs exactes figurent dans `brand-guidelines.md`. Le tableau ci-dessous en donne la structure et la nature : c'est elle qui détermine si une contrainte se vérifie par programme ou demande un jugement.

Groupe	Ce que la charte fixe	Nature
Palette	Couleurs autorisées et leurs usages	Déterministe
Géométrie	viewBox, grille, marge de sécurité	Déterministe
Traits	Largeur, terminaisons, jonctions	Déterministe
Éléments interdits	Primitives, effets et attributs proscrits	Déterministe
Composition	Densité, nombre de formes, niveau de détail	Qualitative
Style	Vocabulaire formel commun aux cinq références	Qualitative

Ce que le jury fera varier

Les fichiers du dépôt servent au développement, aux démonstrations et à vos évaluations internes. À l'épreuve finale, **deux choses changeront**, et aucune n'est connue à l'avance : les concepts soumis, et la charte elle-même.

Indice crucial — les deux axes de généralisation

Concepts secrets : ils relèvent de domaines différents de ceux des cinq références. Mesurer sa cohérence sur les seuls concepts publics donne une estimation **optimiste et trompeuse** — on a appris un catalogue, pas un langage visuel.

Charte secrète : le jury remplacera `brand-guidelines.md` et `references/` aux mêmes emplacements, avant de lancer votre adaptateur. Une palette ou une largeur de trait écrite en dur fera échouer toute la collection : votre système doit **lire sa charte à l'exécution**.

Constituez donc vos propres concepts de contrôle, et fabriquez une charte variante pour vous tester.

3. Votre mission

Votre objectif est de construire une chaîne de génération **automatisée et reproductible**, capable de produire une icône juste et conforme pour n'importe quel concept, et de démontrer que ce qu'elle a appris du langage visuel tient encore sur des concepts qu'elle n'a jamais vus.

L'architecture est **totalelement libre** : rien n'impose de faire produire du SVG directement par un modèle de langage. Représentation intermédiaire JSON, *scene graph*, DSL graphique, composants paramétriques, agents, validateurs, recherche dans un espace de variantes, méthodes symboliques — toute combinaison est recevable, à condition d'être justifiée par une démarche d'AI Engineering reproductible.

Étape 1 — Extraction du langage visuel, à l'exécution

Faites *lire* par votre programme la charte et les cinq SVG de référence, et produire une spécification explicite : palette, grille, largeur de trait, rayons, densité, vocabulaire de formes. Cette spécification est la pièce dont dépend tout le reste — et elle doit être **calculée depuis les fichiers**, jamais recopiée dans le code.

Étape 2 — Chaîne minimale obligatoire

Construisez d'abord le chemin le plus court : un concept en entrée, un fichier `<id>.svg` valide en sortie, de bout en bout. Faites-le tourner avant toute sophistication. Une architecture élégante qui ne produit rien à 16h00 ne vaut rien.

Étape 3 — Validation déterministe

Implémentez les contraintes vérifiables par programme : XML bien formé, viewBox attendue, palette respectée, éléments interdits absents. Un SVG qui échoue à ces contrôles est compté comme non produit — traitez-les donc en premier.

Étape 4 — Évaluation qualitative et cohérence de collection

Les contraintes de sens, de simplicité, de lisibilité et de style ne se vérifient pas par une simple règle. Définissez, pondérez et implémentez une fonction de cohérence sur l'ensemble de la collection :

$$\text{Consistency}(S) = f(\text{palette, trait, densité, géométrie, complexité, style})$$

Vos choix sont libres, mais ils doivent être explicités et testés. L'évaluation du set complet est obligatoire : une icône peut être bonne isolément et fautive dans la famille.

Étape 5 — Boucle d'amélioration

Mettez en place une boucle Generate → Evaluate → Refine. Chaque gain annoncé doit être démontré expérimentalement, par une mesure avant/après sur un jeu de concepts fixé — pas par une impression visuelle.

Étape 6 — Adaptateur et soumission

Exposez votre système derrière le contrat d'exécution imposé au paragraphe suivant. Pour chaque requête du fichier d'entrée, le système doit créer **exactement un** fichier <id>.svg dans le dossier de sortie.

4. Livrables — deadline 16h00

Règle de dépôt

Tout doit être poussé sur votre dépôt GitHub **avant 16h00**. Le dernier commit antérieur à la deadline fera foi.

1. L'adaptateur exécutable

Le dépôt doit fournir une commande unique, exécutable depuis sa racine, dont l'entrée respecte le schéma `benchmark/request.schema.json` :

```
1 python generate.py --input requests.json --output outputs/
```

Une autre commande est admise si votre stack l'exige, à condition de fournir un script `run.sh` ou `run.bat` offrant **exactement le même contrat** : mêmes options, mêmes fichiers produits.

2. Les sorties publiques et le manifeste

Le dépôt contient les SVG générés pour *tous* les concepts publics, ainsi qu'un `manifest.json` décrivant l'environnement et la méthode :

```
1 {
2   "equipe": ["Nom Prenom", "..."],
3   "methode": "representation intermediaire JSON + validateurs + boucle refine",
4   "modeles": [{"nom": "...", "version": "...", "local": true}],
5   "bibliotheques": [...],
6   "services_distants": [],
```

```
7 "repli_gratuit": "decrire la solution locale de secours"  
8 }
```

3. Vidéo de présentation — 3 à 5 minutes

Présentez l'équipe, la spécification du langage visuel que vous avez extraite, votre architecture, votre stratégie d'évaluation, un échec instructif et ce que vous feriez avec une journée de plus.

4. Code et rapport README .md

Code source complet et un **unique** README .md à la racine du dépôt, complété à partir du canevas fourni : installation et exécution, stratégie d'évaluation des contraintes *hard* et *soft*, résultats des évaluations internes, liste exacte des modèles, bibliothèques, données et services utilisés, et responsabilités de chaque membre de l'équipe.

5. Règles du concours

Équité

- Le challenge doit pouvoir être réalisé **sans compte ni API payante**.
- Un service payant n'apporte aucun bonus dans la notation.
- L'évaluation doit pouvoir être menée sans fournir de moyen de paiement.
- Une équipe utilisant un service distant doit documenter une solution de repli gratuite ou locale, et cette solution doit fonctionner.

Automatisation

Ce qui disqualifie une soumission

Dessiner manuellement les icônes demandées n'est pas une solution recevable. Coder des sorties spécifiquement pour une liste supposée de concepts secrets non plus. Toute intervention manuelle dans la chaîne de génération doit être déclarée : la dissimuler est une fraude.

Éthique et propriété

- Ne présenter le projet comme affilié à aucune marque existante.
- Ne redessiner ni intégrer aucun logo de marque dans les icônes produites.
- Respecter les licences des modèles, bibliothèques et assets employés.
- Signaler toute ressource externe.

6. Critères d'évaluation

Les quatre premiers critères sont mesurés par le jury en **exécutant votre dépôt** après la deadline, sur des concepts secrets et sous une charte de remplacement. Les autres portent sur ce que vous remettez.

Critère	Poids
Conformité technique — validation déterministe des SVG produits	20 %
Fidélité sémantique et lisibilité à petite taille	20 %
Généralisation aux concepts secrets	15 %
Généralisation à une charte secrète	10 %
Architecture et boucle d'évaluation	10 %
Cohérence de la collection	10 %
Qualité du code et reproductibilité	5 %
Présentation (vidéo)	10 %
Total	100 %

Les deux lignes de **généralisation** mesurent un *écart*, pas un score absolu. Un système qui a appris un catalogue, ou qui porte sa charte dans son code, s'y effondre de lui-même.

7. Repères d'organisation de la journée

Le découpage ci-dessous est indicatif : il n'est pas évalué et chaque équipe reste libre de son organisation. Il sert de garde-fou pour éviter le piège classique du hackathon — passer sept heures sur l'architecture et trente minutes sur les livrables.

Créneau	Objectif
08h00 – 09h00	Prise en main du dépôt et de la charte, exécution à blanc de l'adaptateur
09h00 – 10h30	Spécification du langage visuel, cadrage de l'architecture, répartition des rôles
10h30 – 11h30	Chaîne minimale : un concept, un SVG valide de bout en bout
11h30 – 13h00	Validateurs déterministes, premier passage sur tous les concepts publics
13h00 – 14h30	Cohérence de collection, boucle Generate → Evaluate → Refine
14h30 – 15h15	Contrôle sur vos concepts réservés et sur votre charte variante
15h15 – 15h45	Génération finale du set public, manifest.json et README.md
15h45 – 16h00	Vérifications finales et dernier commit

Enregistrez la vidéo dès que l'architecture est figée : c'est le livrable le plus souvent sacrifié faute de temps.

Avant de pousser votre dernier commit

- Le dépôt cloné dans un dossier vierge produit un <id>.svg par requête.
- Chaque SVG est bien formé, respecte la viewBox et la palette, sans élément interdit.
- Le système a été exécuté au moins une fois sur des concepts *hors* du domaine des références.
- Il a été exécuté au moins une fois avec une charte modifiée, sans rien changer au code.
- manifest.json liste modèles, bibliothèques et services réellement utilisés.
- Les scores annoncés dans le README.md sont reproductibles, graines fixées.
- La vidéo est accessible (lien vérifié) et dure de 3 à 5 minutes.